Practical 1

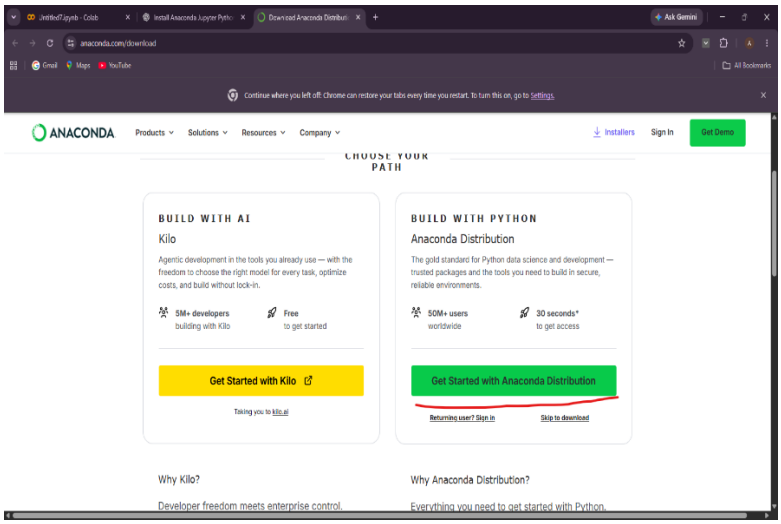
Aim:- To install Anaconda, launch Jupyter Notebook, and execute a Hello World program in Python to verify the successful installation and working of the Python environment.

Background Information:-

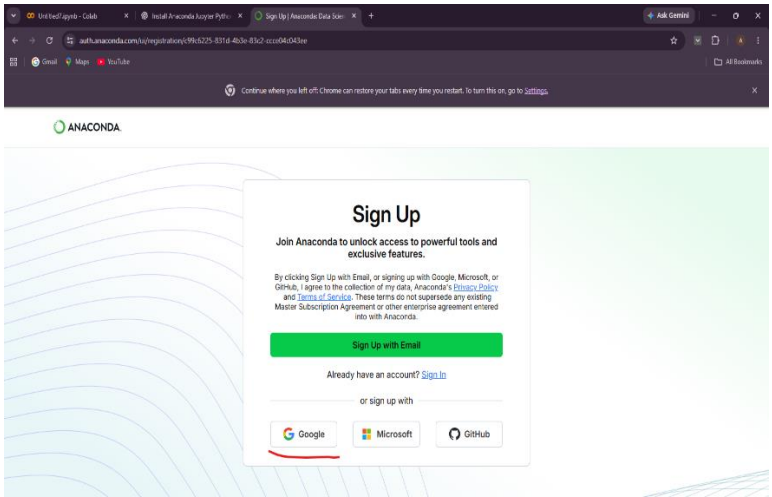
- Anaconda is a Python distribution used for programming and data analysis.
- Jupyter Notebook is an interactive environment used to write and execute Python code.
- Python code is written in cells and executed to display the output.
- The Hello World program is used to verify that Python is installed and working correctly.

Steps:-

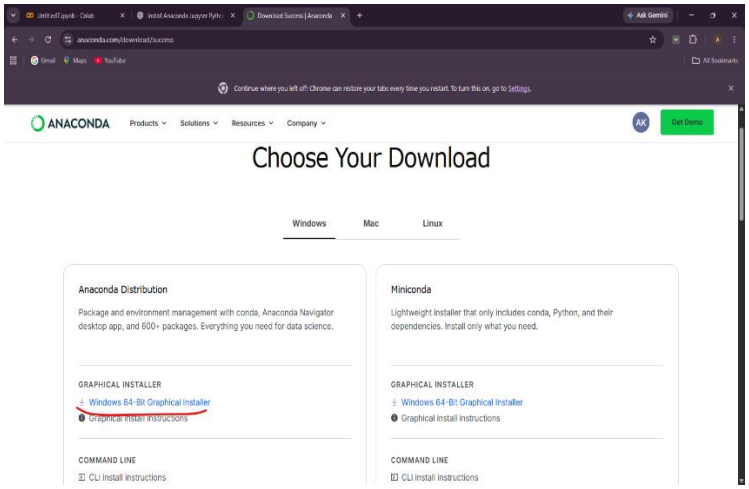
1. Open the official Anaconda website and navigate to the Anaconda distribution download page.



2. Create an Anaconda account or sign in to continue with the download, if required.



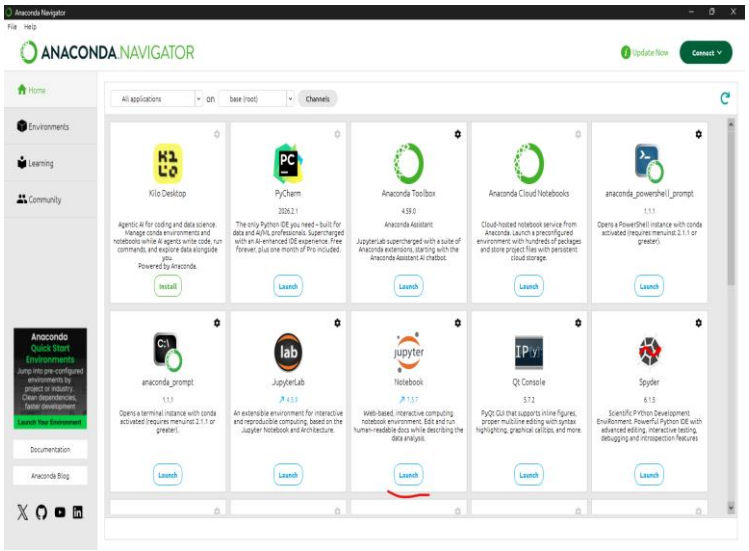
3. Open the Anaconda download page and select the appropriate installer for Windows.



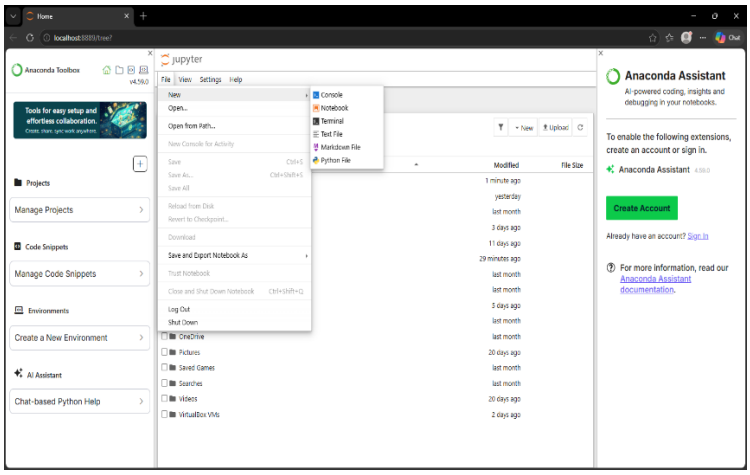
4. Run the downloaded Anaconda installer and click Next to begin the installation process.



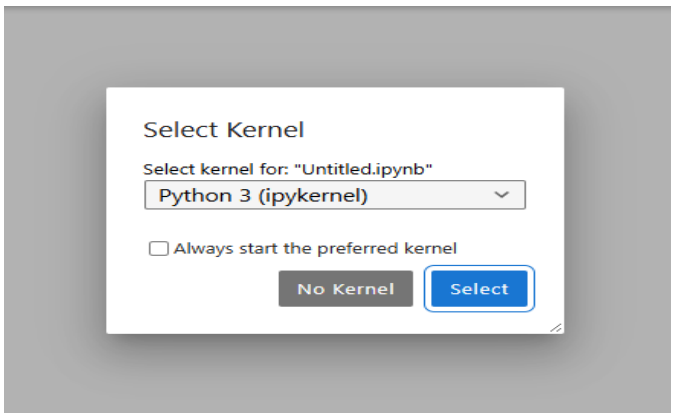
5. After installation, open Anaconda Navigator. It provides access to applications such as Jupyter Notebook and other Python tools.



6. Open Jupyter Notebook from Anaconda Navigator and navigate to the required folder. Click New → Notebook to create a new notebook.

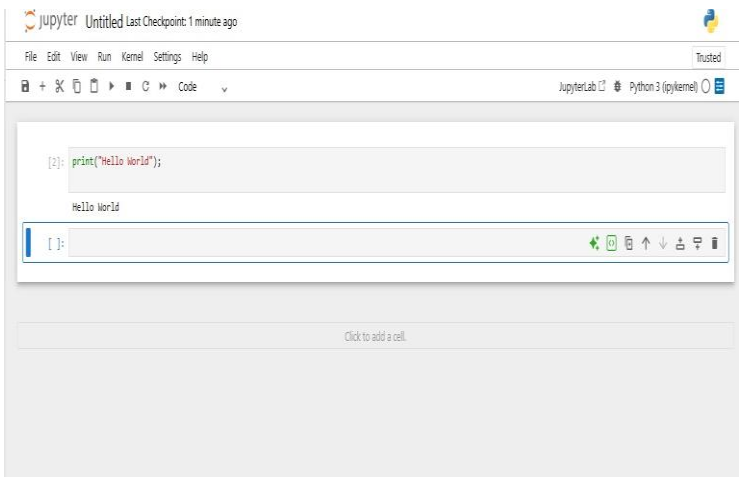


7. In the Select Kernel window, choose Python 3 (ipykernel) and click Select to connect the notebook to Python.



8. Enter the following code in the notebook cell and press Shift + Enter to execute it. Verify that the output displays `Hello World`.

   - print("Hello World")



PRACTICAL 2:

Simple Malware Detection using Decision Tree and Scikit-learn

Aim

To build a simple machine learning model using a Decision Tree Classifier in Scikit-learn to classify files as benign (safe) or malware (malicious) based on their features.

Background Information

A Decision Tree is a supervised machine learning algorithm that makes predictions by learning rules from labelled data. In malware detection, it can analyse features such as file size, number of imported functions, and suspicious API calls to classify a file as benign or malicious.

Scikit-learn provides the DecisionTreeClassifier class to build this model. In this practical, a small synthetic dataset is used to demonstrate the process. This is a learning example, not a reliable real-world malware scanner.

Requirements

1. Google Colab
2. Python
3. NumPy
4. Pandas
5. Scikit-learn
6. Matplotlib (for visualising the Decision Tree)

Dataset Description

Feature	**Meaning**
file_size	Size of the file
suspicious_calls	Number of suspicious API calls
entropy	Measure of randomness in the file
malware	Target label: 0 = Benign, 1 = Malware

Procedure

1. Open a browser and visit Google Colab.
2. Sign in using your Google account.

3. Click on New Notebook.
4. Import the required Python libraries.
5. Create a sample dataset containing file features and labels.
6. Separate the features (X) from the target labels (y).
7. Split the dataset into training and testing sets.
8. Create a Decision Tree Classifier.
9. Train the model using the training data.
10. Predict whether the test samples are benign or malware.
11. Evaluate the model using accuracy and a classification report.
12. Predict the class of a new file.
13. Visualise the trained Decision Tree.

Implementation: Python Code

Copy the following code into a single Google Colab cell and click Run.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report

# Step 2: Generate the Dataset
np.random.seed(42)
n = 20

data = pd.DataFrame({
    'file_size': np.random.randint(10, 1000, n),
```

```
    'suspicious_calls': np.random.randint(0, 30, n),
    'entropy': np.random.uniform(2, 8, n)
})

# Create labels: 0 = Benign, 1 = Malware
data['malware'] = (
    (data['suspicious_calls'] > 15) |
    (data['entropy'] > 5.5)
).astype(int)

print("Dataset:")
print(data)

# Step 3: Separate Features and Target
X = data[['file_size', 'suspicious_calls', 'entropy']]
y = data['malware']

# Step 4: Split the Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 5: Create and Train the Decision Tree
model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
```

```
print("\nModel trained successfully!")

# Step 6: Predict and Evaluate the Model
y_pred = model.predict(X_test)

print("\nPredicted values:", y_pred)
print("Actual values:", y_test.values)

accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100, "%")

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    labels=[0, 1],
    target_names=['Benign', 'Malware'],
    zero_division=0
))

# Step 7: Predict a New File
new_file = pd.DataFrame(
    [[750, 20, 7.0]],
    columns=['file_size', 'suspicious_calls', 'entropy']
)

prediction = model.predict(new_file)
```

```
if prediction[0] == 1:
    print("\nNew File Prediction: Malware")
else:
    print("\nNew File Prediction: Benign")

# Step 8: Visualize the Decision Tree
plt.figure(figsize=(14, 8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=['Benign', 'Malware'],
    filled=True,
    rounded=True
)

plt.title("Decision Tree for Malware Detection")
plt.show()
```

Results:

The Decision Tree successfully learned from the given dataset and classified the test records into benign and malware categories. The model accuracy is displayed after running the evaluation code. The new sample file was also classified by the trained Decision Tree model.

Conclusion:

The practical successfully demonstrated the implementation of a simple malware detector using a Decision Tree and Scikit-learn. The model was trained using file size, suspicious API calls, and entropy, and was able to classify records as benign or malware

PRACTICAL 4:

Network Anomaly Detection using Decision Tree and Isolation Forest

Aim

To detect abnormal (attack) network traffic using (a) a supervised machine learning classifier, the Decision Tree, and (b) an unsupervised anomaly detection algorithm, the Isolation Forest, and to compare how both perform.

Background / Theory

1. Network Anomaly Detection

Network anomaly detection identifies network traffic that differs from normal patterns. Attacks like DoS (Denial of Service) and port scanning can create unusually high numbers of connections or unusual data transfers.

2. Decision Tree

A Decision Tree is a supervised machine learning algorithm that learns from labelled data and classifies traffic as normal or attack using feature-based decisions.

3. Isolation Forest

Isolation Forest is an unsupervised algorithm that does not require labels for training. It uses random trees to isolate unusual records. Anomalies are isolated faster with shorter paths.

Requirements

1. Google Colab
2. Python
3. NumPy
4. Pandas
5. Matplotlib
6. Scikit-learn

Procedure

1. Open Google Colab and create a new notebook.
2. Import the required libraries.
3. Create sample network traffic data containing duration, bytes sent, connections, and labels.
4. Visualise the normal and attack traffic using a scatter plot.

5. Split the labelled data into training and testing sets.
6. Train a Decision Tree Classifier and evaluate its predictions.
7. Train an Isolation Forest to detect anomalies without using labels.
8. Plot the anomalies detected by Isolation Forest.

Implementation: Complete Python Code

Copy this code into one Google Colab cell and run it.

```
# PRACTICAL 3: Network Anomaly Detection
# Using Decision Tree and Isolation Forest

# Step 1: Import the Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import IsolationForest
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# Step 2: Create the Sample Network Traffic Data
np.random.seed(42)

# 50 normal network traffic records
normal = pd.DataFrame({
    'duration': np.random.normal(5, 1.5, 50),
    'bytes_sent': np.random.normal(500, 100, 50),
    'connections': np.random.normal(10, 3, 50),
```

```
    'label': 0
})

# 25 attack network traffic records
attack = pd.DataFrame({
    'duration': np.random.normal(30, 5, 25),
    'bytes_sent': np.random.normal(5000, 800, 25),
    'connections': np.random.normal(120, 20, 25),
    'label': 1
})

# Combine normal and attack records
data = pd.concat([normal, attack], ignore_index=True)

print("Sample Network Traffic Data:")
print(data.head())

print("\nLabel Counts:")
print(data['label'].value_counts())

# Step 3: Visualize the Data
colors = data['label'].map({0: 'blue', 1: 'red'})

plt.scatter(
    data['bytes_sent'],
    data['connections'],
    c=colors,
    alpha=0.8
```

```
)

plt.xlabel('Bytes Sent')
plt.ylabel('Connections')
plt.title('Blue = Normal, Red = Attack')
plt.show()

# Step 4: Split the Data
X = data[['duration', 'bytes_sent', 'connections']]
y = data['label']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Step 5: Decision Tree Classifier
clf = DecisionTreeClassifier(random_state=42)

clf.fit(X_train, y_train)

pred = clf.predict(X_test)

print("\nDecision Tree Confusion Matrix:")
print(confusion_matrix(y_test, pred))
```

```
print("\nDecision Tree Classification Report:")
print(classification_report(
    y_test,
    pred,
    labels=[0, 1],
    target_names=['Normal', 'Attack'],
    zero_division=0
))

# Step 6: Isolation Forest
iso = IsolationForest(
    n_estimators=100,
    contamination=0.33,
    random_state=42
)

# Train without using labels
iso.fit(X)

# Isolation Forest: 1 = Normal, -1 = Anomaly
data['anomaly'] = iso.predict(X)

# Convert predictions to 0 = Normal, 1 = Anomaly
data['anomaly'] = data['anomaly'].map({
    1: 0,
    -1: 1
})
```

```
print("\nIsolation Forest Confusion Matrix:")
print(confusion_matrix(data['label'], data['anomaly']))

print("\nIsolation Forest Classification Report:")
print(classification_report(
    data['label'],
    data['anomaly'],
    labels=[0, 1],
    target_names=['Normal', 'Attack'],
    zero_division=0
))

# Step 7: Plot Detected Anomalies
colors = data['anomaly'].map({0: 'blue', 1: 'red'})

plt.scatter(
    data['bytes_sent'],
    data['connections'],
    c=colors,
    alpha=0.8
)

plt.xlabel('Bytes Sent')
plt.ylabel('Connections')
plt.title('Isolation Forest: Red = Detected Anomaly')
plt.show()
```

Conclusion

Network anomaly detection was successfully performed using Decision Tree and Isolation Forest in Google Colab. The Decision Tree classified the labelled normal and attack traffic, while Isolation Forest detected anomalies without using labels. Decision Tree is suitable when labelled attack data is available, whereas Isolation Forest is useful when labelled data is unavailable and new or unknown attacks need to be detected.

PRACTICAL 5:

Face-Lock System using PCA and KNN

Aim

To create a simple face-lock system using PCA and KNN, where authorized faces are enrolled and new faces are accepted or rejected based on a distance threshold. The system's FAR (False Acceptance Rate) and FRR (False Rejection Rate) are also evaluated.

Background / Theory

1. Facial Recognition: Identifies a person using facial features.
2. PCA (Principal Component Analysis): Reduces face image data into important features.
3. KNN (K-Nearest Neighbors): Finds the closest matching enrolled face.
4. Threshold: Decides whether to accept or reject a face based on its distance from an enrolled face.
5. FAR (False Acceptance Rate): Measures how often an unauthorized person is incorrectly accepted.
6. FRR (False Rejection Rate): Measures how often an authorized person is incorrectly rejected.

Requirements

1. Google Colab
2. Python
3. NumPy
4. Matplotlib
5. Scikit-learn

Procedure

1. Open Google Colab and create a new notebook.
2. Import the required libraries.
3. Load the Olivetti Faces dataset.
4. Enrol authorized users into the face-lock system.
5. Apply PCA to extract important facial features.
6. Use KNN to find the closest matching enrolled face.

7. Set a threshold to accept or reject a face.
8. Calculate FAR and FRR.
9. Test authentication using an authorized user and an impostor.

Implementation: Complete Python Code

Copy the complete code into one Google Colab cell and click Run. The first run may take some time because the face dataset needs to be downloaded.

```
# Step 1: Import Libraries

import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import fetch_olivetti_faces

from sklearn.decomposition import PCA

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score


# Step 2: Load the Face Dataset

faces = fetch_olivetti_faces(shuffle=False)


X = faces.data

y = faces.target


print("Dataset loaded successfully")

print("Total face images:", X.shape[0])

print("Number of people:", len(np.unique(y)))


# Step 3: Enroll Authorized Users

# People 0-29 are authorized.
```

```
# People 30-39 are treated as impostors.

authorized_ids = np.arange(0, 30)
impostor_ids = np.arange(30, 40)

# Use first 5 images per authorized person for enrollment
train_indices = []
calibration_indices = []
test_indices = []
impostor_indices = []

for person in authorized_ids:
    person_indices = np.where(y == person)[0]

    train_indices.extend(person_indices[:5])
    calibration_indices.extend(person_indices[5:7])
    test_indices.extend(person_indices[7:])

for person in impostor_ids:
    person_indices = np.where(y == person)[0]
    impostor_indices.extend(person_indices)

X_train = X[train_indices]
y_train = y[train_indices]

X_cal = X[calibration_indices]
y_cal = y[calibration_indices]
```

```
X_test = X[test_indices]
y_test = y[test_indices]

X_impostor = X[impostor_indices]

print("\nAuthorized users enrolled:", len(authorized_ids))
print("Enrollment images:", len(X_train))
print("Calibration images:", len(X_cal))
print("Authorized test images:", len(X_test))
print("Impostor test images:", len(X_impostor))

# Step 4: PCA + KNN
# PCA reduces the number of facial features.
pca = PCA(n_components=100, whiten=True, random_state=42)

X_train_pca = pca.fit_transform(X_train)
X_cal_pca = pca.transform(X_cal)
X_test_pca = pca.transform(X_test)
X_impostor_pca = pca.transform(X_impostor)

# KNN finds the nearest enrolled face.
knn = KNeighborsClassifier(n_neighbors=1, metric='euclidean')
knn.fit(X_train_pca, y_train)

print("\nPCA and KNN models trained successfully")

# Step 5: Set the Acceptance Threshold
# Calibrate the threshold using genuine authorized faces.
```

```
cal_distances, cal_neighbors = knn.kneighbors(X_cal_pca)

threshold = np.percentile(cal_distances[:, 0], 95)

print("\nAcceptance Threshold:", round(threshold, 3))

# Step 6: Test FAR and FRR
test_distances, test_neighbors = knn.kneighbors(X_test_pca)
impostor_distances, impostor_neighbors = knn.kneighbors(
    X_impostor_pca
)

# Accept only when the closest distance is within the threshold.
authorized_accepted = test_distances[:, 0] <= threshold
impostor_accepted = impostor_distances[:, 0] <= threshold

# FRR: Authorized faces incorrectly rejected.
FRR = np.mean(~authorized_accepted) * 100

# FAR: Impostor faces incorrectly accepted.
FAR = np.mean(impostor_accepted) * 100

print("\nFalse Acceptance Rate (FAR):", round(FAR, 2), "%")
print("False Rejection Rate (FRR):", round(FRR, 2), "%")

# Optional: Identity classification accuracy
predicted_ids = knn.predict(X_test_pca)
print(
```

```
    "\nNearest-neighbour identity accuracy:",
    round(accuracy_score(y_test, predicted_ids) * 100, 2),
    "%"
)

# Step 7: Authenticate an Authorized User and an Impostor
def authenticate(face_image, description):
    face_pca = pca.transform(face_image.reshape(1, -1))

    distances, neighbors = knn.kneighbors(face_pca)
    distance = distances[0, 0]
    matched_id = knn.predict(face_pca)[0]

    print("\nTesting:", description)
    print("Nearest enrolled user ID:", matched_id)
    print("Distance:", round(distance, 3))

    if distance <= threshold:
        print("Access Granted")
    else:
        print("Access Denied")

# Test an authorized user's held-out image
authenticate(X_test[0], "Authorized User")

# Test an impostor's image
authenticate(X_impostor[0], "Impostor")
```

```
# Display sample face images
plt.figure(figsize=(8, 3))

plt.subplot(1, 2, 1)
plt.imshow(X_test[0].reshape(64, 64), cmap='gray')
plt.title("Authorized User")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(X_impostor[0].reshape(64, 64), cmap='gray')
plt.title("Impostor")
plt.axis('off')

plt.tight_layout()
plt.show()
```

Conclusion

The facial recognition-based authentication system was implemented using PCA for feature extraction and 1-Nearest Neighbor for face matching. The system identifies authorized users and rejects faces whose distance exceeds the selected threshold. FAR and FRR are evaluated to understand the system's authentication errors. A suitable threshold is important to balance false acceptance and false rejection and improve reliability.

Practical 6 –

KNeighborsClassifier-Based Authentication

Aim - To authenticate users (genuine user or imposter) using the KNeighborsClassifier algorithm in Python.

Background Information –

1. Authentication is used to verify a user's identity before giving access to a system.
2. KNN is a machine learning algorithm used to classify data.
3. It checks typing speed, key hold time, and key gap time to identify genuine users and imposters.
4. The model displays Access Granted for a genuine user and Access Denied for an imposter.

Steps –

1. **Open Google Chrome and visit https://colab.research.google.com/.**
2. **Sign in using your Google account.**
3. **Click on File → New notebook to create a notebook**
4. **Run the code in Google Collab**

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report



np.random.seed(42)

genuine = pd.DataFrame({

    'typing_speed': np.random.normal(60, 5, 200),

    'key_hold_time': np.random.normal(0.15, 0.02, 200),

    'key_gap_time': np.random.normal(0.30, 0.04, 200),

    'label': 1})
```

```
imposter = pd.DataFrame({
    'typing_speed': np.random.normal(45, 8, 200),
    'key_hold_time': np.random.normal(0.22, 0.04, 200),
    'key_gap_time': np.random.normal(0.45, 0.08, 200),
    'label': 0})
data = pd.concat([genuine, imposter]).reset_index(drop=True)
print(data.head())

X = data.drop('label', axis=1)
y = data['label']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

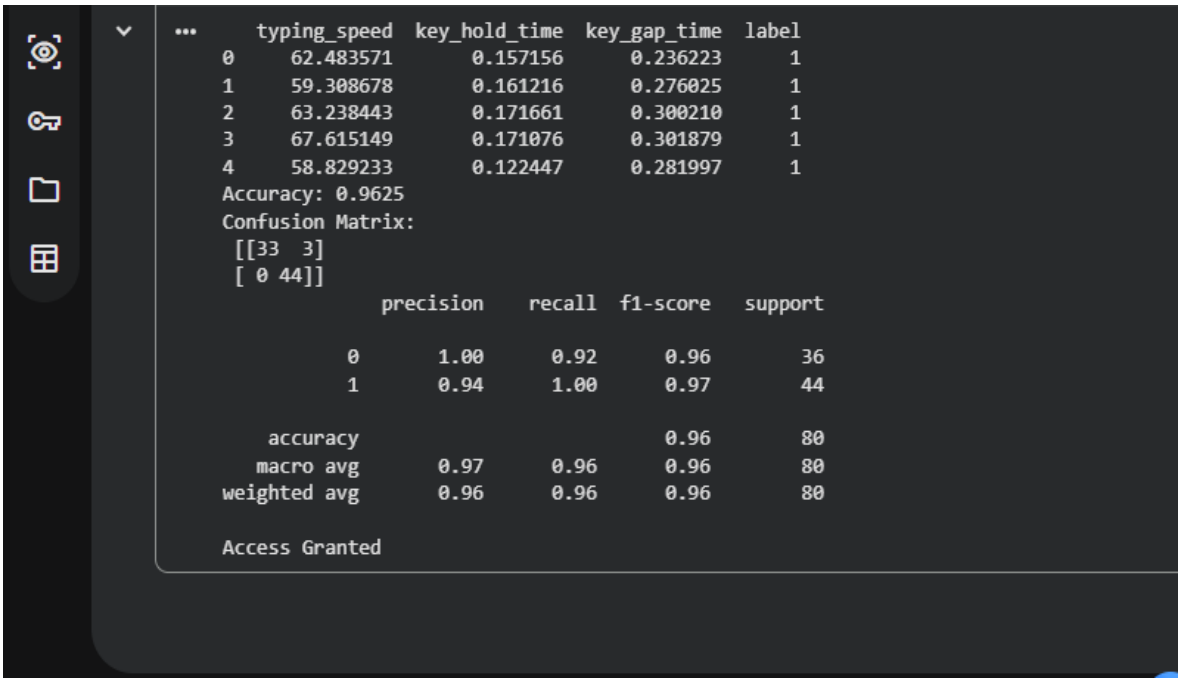
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
new_attempt = pd.DataFrame([[58, 0.16, 0.31]], columns=X.columns)

result = knn.predict(scaler.transform(new_attempt))

print("Access Granted" if result[0] == 1 else "Access Denied")
```

Output –



```
    typing_speed  key_hold_time  key_gap_time  label
0      62.483571       0.157156      0.236223      1
1      59.308678       0.161216      0.276025      1
2      63.238443       0.171661      0.300210      1
3      67.615149       0.171076      0.301879      1
4      58.829233       0.122447      0.281997      1
Accuracy: 0.9625
Confusion Matrix:
 [[33  3]
 [ 0 44]]
              precision    recall  f1-score   support

           0       1.00      0.92      0.96        36
           1       0.94      1.00      0.97        44

    accuracy                           0.96        80
   macro avg       0.97      0.96      0.96        80
weighted avg       0.96      0.96      0.96        80

Access Granted
```

Result -

The KNeighborsClassifier model was trained on 320 samples and tested on 80 samples. It reached an accuracy of 96.25%. It accepted a new login attempt with genuine-user behaviour (Access Granted).

Conclusion –

KNN can be used to authenticate users based on their typing behaviour. It separated genuine users from imposters with high accuracy. In the confusion matrix, no genuine user was rejected, but 3 imposters were wrongly accepted. In a real security system such errors must be kept very low, which can be improved with more data, more features, or a better value of K

PRACTICAL 7
Fraud Detection Using a Sample Dataset and ML Models

Aim:-

To detect fraudulent transactions using machine learning models (Logistic Regression and Random Forest) on a sample dataset, and to compare the performance of the two models.

Background Information:-

1. Fraud Detection: Identifies fraudulent or suspicious transactions.
2. Importance: Helps prevent stolen-card payments and unusual transfers.
3. Dataset and Features: Uses 2,000 Python-generated transactions with amount, time, location risk, device risk, transaction count, and balance change.
4. Target Class: Class 0 – Normal; Class 1 – Fraud.
5. Algorithms: Logistic Regression and Random Forest are used to detect fraud.
6. Evaluation and Objective: Compares both models using accuracy, precision, recall, F1-score, and confusion matrix.

Steps:-

1. **Open Google Chrome and visit https://colab.research.google.com/.**
2. **Sign in using your Google account.**
3. **Click on File → New notebook to create a notebook**
4. **Run the code in Google Collab**

Code:-

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Step 2: Generate Sample Dataset
np.random.seed(42)

n_samples = 2000

data = pd.DataFrame({
    'amount': np.random.uniform(10, 10000, n_samples),
    'transaction_hour': np.random.randint(0, 24, n_samples),
```

```
    'location_mismatch': np.random.randint(0, 2, n_samples),
    'international': np.random.randint(0, 2, n_samples),
    'previous_fraud': np.random.randint(0, 2, n_samples),
    'transaction_frequency': np.random.randint(1, 20, n_samples)
})

# Create fraud labels
fraud_probability = (
    0.02
    + 0.15 * data['location_mismatch']
    + 0.20 * data['international']
    + 0.30 * data['previous_fraud']
    + 0.01 * (data['transaction_frequency'] > 15)
)

fraud_probability = np.clip(fraud_probability, 0, 0.90)

data['fraud'] = (
    np.random.random(n_samples) < fraud_probability
).astype(int)

print("Dataset Sample:")
print(data.head())

print("\nClass Distribution:")
print(data['fraud'].value_counts())

# Step 3: Separate Features and Target
X = data.drop('fraud', axis=1)
y = data['fraud']

# Step 4: Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

# Step 5: Feature Scaling for Logistic Regression
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Step 6: Train Logistic Regression Model
lr_model = LogisticRegression(class_weight='balanced')

lr_model.fit(X_train_scaled, y_train)
```

```
lr_predictions = lr_model.predict(X_test_scaled)

print("\n--- Logistic Regression Results ---")
print("Accuracy:", accuracy_score(y_test, lr_predictions))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, lr_predictions))

print("\nClassification Report:")
print(classification_report(
    y_test,
    lr_predictions,
    zero_division=0
))

# Step 7: Train Random Forest Model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

rf_model.fit(X_train, y_train)

rf_predictions = rf_model.predict(X_test)

print("\n--- Random Forest Results ---")
print("Accuracy:", accuracy_score(y_test, rf_predictions))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, rf_predictions))

print("\nClassification Report:")
print(classification_report(
    y_test,
    rf_predictions,
    zero_division=0
))

# Step 8: Compare Both Models
comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [
        accuracy_score(y_test, lr_predictions),
        accuracy_score(y_test, rf_predictions)
    ]
})

print("\n--- Model Comparison ---")
print(comparison)

# Step 9: Result
print("\nResult:")
print("Fraud detection models were trained and evaluated successfully.")
```

```
print("Both models were compared using accuracy and classification reports.")

# Step 10: Conclusion
print("\nConclusion:")
print("Machine learning can help identify potentially fraudulent transactions.")
print("The model's performance should be evaluated using fraud precision,")
print("recall, F1-score, and the confusion matrix, not accuracy alone.")
```

Output :-

```
First Five Rows:
   amount  time  location_risk  device_risk  num_transactions  balance_change  \
0   -3.01 -0.17          -1.37         0.22             -2.26           -2.78
1   -1.30 -0.17          -2.68        -0.75             -3.32           -0.93
2    2.29 -1.16          -1.51        -1.72             -2.79            1.35
3    1.88  0.18          -1.62        -1.34             -0.18            4.04
4    0.19 -1.91          -3.77         0.37              0.64            3.34

   Class
0      0
1      0
2      0
3      0
4      0

Class Counts:
Class
0    1892
1     108
Name: count, dtype: int64

Training Rows: 1600
Testing Rows: 400

--- LOGISTIC REGRESSION ---
Accuracy: 0.815
Confusion Matrix:
[[310  68]
 [  6  16]]
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.82      0.89       378
           1       0.19      0.73      0.30        22

    accuracy                           0.81       400
   macro avg       0.59      0.77      0.60       400
weighted avg       0.94      0.81      0.86       400
```

```
--- RANDOM FOREST ---
Accuracy: 0.975
Confusion Matrix:
[[378   0]
 [ 10  12]]
Classification Report:
              precision    recall  f1-score   support

           0       0.97      1.00      0.99       378
           1       1.00      0.55      0.71        22

    accuracy                           0.97       400
   macro avg       0.99      0.77      0.85       400
weighted avg       0.98      0.97      0.97       400


--- MODEL COMPARISON ---
              Model  Accuracy  Fraud Precision  Fraud Recall  Fraud F1-Score
Logistic Regression     0.815             0.19         0.727           0.302
      Random Forest     0.975             1.00         0.545           0.706

--- RESULT ---
Both models trained and tested successfully.

--- CONCLUSION ---
Machine learning models can detect suspicious transactions in sample data. Accuracy, precision, recall and F1-score help evaluate their performance.
```

Result

A synthetic dataset of 2,000 transactions was generated and divided into 1,600 training rows and 400 testing rows. Both Logistic Regression and Random Forest were trained and evaluated. The visible output shows Logistic Regression accuracy of 81.5%; add the remaining metrics from the full output in your notebook.

Conclusion

This practical demonstrates how machine-learning models classify sample transactions as normal or fraudulent. Since fraud cases are rare, precision, recall, F1-score and the confusion matrix should be checked along with accuracy. These results are based on synthetic practice data and do not represent real banking performance.

PRACTICAL – 7A

Evaluating Detector Performance: Feature Engineering

Aim : To evaluate how feature engineering (creating new, more useful features from the raw data) improves the performance of a fraud detector.

Background Information: Feature engineering is the process of creating new features from existing data to improve a machine learning model. In fraud detection, features such as unusual transaction amounts, night-time transactions, and transaction frequency help identify suspicious activities. In this practical, Logistic Regression is evaluated before and after adding four new features using accuracy, precision, recall, and F1-score.

Steps / Procedure:

1. **Open Google Chrome and visit https://colab.research.google.com/.**
2. **Sign in using your Google account.**
3. **Click on File → New notebook to create a notebook.**
4. **Run the code.**

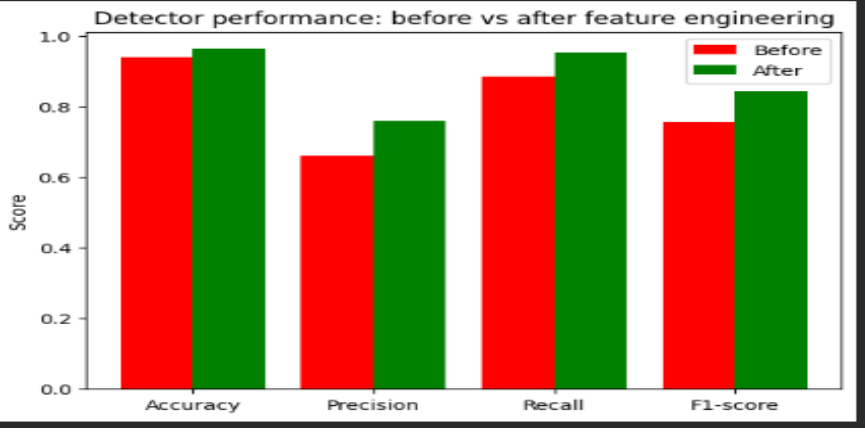
Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
np.random.seed(42)
n = 5000
fraud = np.random.rand(n) < 0.10
avg_amount = np.random.lognormal(4, 0.9, n)
amount = np.where(fraud, avg_amount * np.random.lognormal(0.9, 0.35, n), avg_amount *
np.random.lognormal(0.0, 0.3, n))
hour = np.where(fraud & (np.random.rand(n) < 0.6), np.random.randint(0, 6, n),
np.random.randint(6, 24, n))
distance = np.where(fraud, np.random.exponential(13, n), np.random.exponential(10, n))
txn_24h = np.where(fraud, np.random.poisson(3, n), np.random.poisson(2, n))
data = pd.DataFrame({'amount': amount, 'avg_amount': avg_amount, 'hour': hour,
'distance_km': distance, 'txn_last_24h': txn_24h, 'Class': fraud.astype(int)})
print(data['Class'].value_counts())
def evaluate(df):
    X, y = df.drop('Class', axis=1), df['Class']
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
    sc = StandardScaler()
    X_tr, X_te = sc.fit_transform(X_tr), sc.transform(X_te)
    model = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
```

```
    pred = model.fit(X_tr, y_tr).predict(X_te)
    return [accuracy_score(y_te, pred), precision_score(y_te, pred), recall_score(y_te, pred),
f1_score(y_te, pred)]
before = evaluate(data)
eng = data.copy()
eng['amount_ratio'] = eng['amount'] / eng['avg_amount']
eng['is_night'] = (eng['hour'] < 6).astype(int)
eng['log_amount'] = np.log1p(eng['amount'])
eng['distance_x_txn'] = eng['distance_km'] * eng['txn_last_24h']
after = evaluate(eng)
names = ['Accuracy', 'Precision', 'Recall', 'F1-score']
print(pd.DataFrame({'Before': before, 'After': after}, index=names).round(3))
x = np.arange(4)
plt.bar(x - 0.2, before, 0.4, label='Before', color='red')
plt.bar(x + 0.2, after, 0.4, label='After', color='green')
plt.xticks(x, names)
plt.ylabel('Score')
plt.legend()
plt.title('Detector performance: before vs after feature engineering')
plt.show()
```

Output:

```
Class
0    4475
1     525
Name: count, dtype: int64
           Before  After
Accuracy    0.940  0.963
Precision   0.660  0.758
Recall      0.886  0.952
F1-score    0.756  0.844
```



Conclusion:

Feature engineering improved the performance of the fraud detector without changing the model. Features such as amount_ratio and is_night express fraud patterns directly, so even a simple model can use them. Good features are often as important as the choice of model.